

Data Structures: Queues

Data Structure Course
DGIST

October 30, 2025

Contents

1. Introduction to Queues
2. Core Operations
3. Circular Queue
4. Deque (Double-Ended Queue)
5. Priority Queue
6. Implementation Approaches
7. Applications
8. Complexity Analysis
9. Queue vs Stack
10. Summary

Introduction to Queues

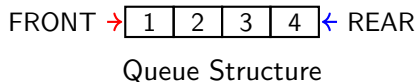
What is a Queue?

Definition

A **queue** is a linear data structure that follows the **First In, First Out (FIFO)** principle.

Key characteristics:

- Elements are inserted at the rear (enqueue)
- Elements are removed from the front (dequeue)
- Perfect for scheduling and buffering
- Models real-world waiting lines
- Used in OS scheduling and network packet handling



Core Operations

Essential Queue Operations

Primary Operations

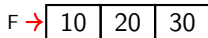
- **Enqueue:** Add element to rear
- **Dequeue:** Remove and return front element
- **Front/Ppeek:** View front element without removing
- **Empty:** Check if queue is empty
- **Size:** Get number of elements

C Example

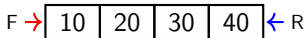
```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5     int data;
6     struct Node* next;
7 } Node;
8
9 typedef struct Queue {
10     Node *front, *rear;
11 } Queue;
12
13 void enqueue(Queue* q, int x) {
14     Node* n = malloc(sizeof(Node));
15     n->data = x; n->next = NULL;
16     if (q->rear) q->rear->next = n;
17     q->rear = n;
18     if (!q->front) q->front = n;
19 }
20
21 int dequeue(Queue* q) {
22     int x = q->front->data;
23     Node* tmp = q->front;
24     q->front = q->front->next;
25     if (!q->front) q->rear = NULL;
26     free(tmp); return x;
27 }
```

Queue Operations Visualization

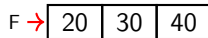
Initial Queue



After Enqueue(40)



After Dequeue()



Returns: 10

Circular Queue

Circular Queue Concept

Why Circular Queue?

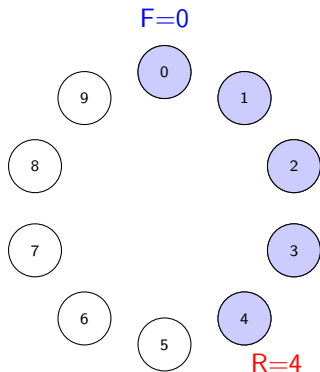
- Simple array queue wastes space
- Front pointer moves forward
- Space at beginning becomes unusable
- Solution: Wrap around using modulo

Wrap-around Formula

$(\text{index} + 1) \% \text{capacity}$

Full/Empty Detection:

- Track size explicitly, OR
- Reserve one empty slot



Circular Buffer

Circular Queue Implementation

```
1 class CircularQueue:
2     def __init__(self, capacity):
3         self.data = [None] * capacity
4         self.capacity = capacity
5         self.front = 0
6         self.size = 0
7
8     def enqueue(self, item):
9         if self.size == self.capacity:
10             raise Exception("Queue is full")
11         rear = (self.front + self.size) % self.capacity
12         self.data[rear] = item
13         self.size += 1
14
15     def dequeue(self):
16         if self.size == 0:
17             raise Exception("Queue is empty")
```

Deque (Double-Ended Queue)

Deque: Double-Ended Queue

Definition

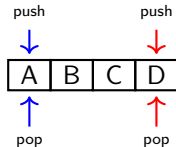
A **deque** (pronounced "deck") allows insertion and deletion at both ends.

Deque Operations

- `pushFront(x)`: Insert at front
- `pushBack(x)`: Insert at rear
- `popFront()`: Remove from front
- `popBack()`: Remove from rear

Use Cases

- Palindrome checking
- Sliding window algorithms
- Browser history (forward/back)



Both ends accessible

Priority Queue

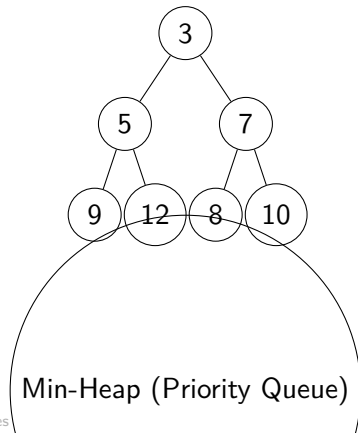
Priority Queue

Definition

A **priority queue** is a queue where elements are served based on priority rather than insertion order.

Key Properties

- Each element has a priority
- Highest (or lowest) priority served first
- Typically implemented with binary heap
- Insert: $O(\log n)$



Implementation Approaches

Array-based vs Linked List Implementation

Feature	Array-based	Linked List-based
Time Complexity	$O(1)$ amortized	$O(1)$ guaranteed
Space Efficiency	Better (contiguous)	More overhead (pointers)
Cache Performance	Excellent	Fair
Resize Cost	Occasional $O(n)$	Never
Capacity	Fixed (or resizable)	Dynamic

Array-based Queue

- Use circular buffer
- Track front and rear indices
- Better for bounded queues

Linked List Queue

- Maintain front and rear pointers
- No capacity concerns
- Better for unbounded queues

Linked List Queue Implementation

```
1 class Node:
2     def __init__(self, val, next=None):
3         self.val = val
4         self.next = next
5
6 class QueueLL:
7     def __init__(self):
8         self.front = None
9         self.rear = None
10        self.size = 0
11
12    def enqueue(self, x):
13        new_node = Node(x)
14        if self.rear:
15            self.rear.next = new_node
16        self.rear = new_node
17        if not self.front:
```

Applications

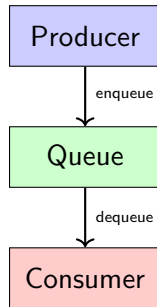
Producer-Consumer Pattern

Bounded Buffer Problem

- Producers generate data
- Consumers process data
- Queue acts as buffer
- Handles rate mismatch

Synchronization:

- If full → producer waits
- If empty → consumer waits
- Use locks/semaphores for thread safety



Examples

Print spooler, web server request handling

Queues in Operating Systems

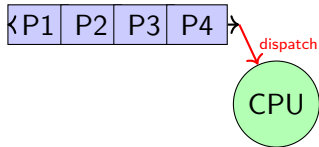
CPU Scheduling

- Ready queue: processes ready to run
- Multiple priority queues
- Round-robin scheduling
- Multi-level feedback queue (MLFQ)

I/O Scheduling

- Disk request queue
- Network packet queue
- Print job queue

Ready Queue



OS Scheduler

Queues in Networking

Network Routers and Switches

- Incoming packets queued before processing
- Multiple queues for Quality of Service (QoS)
- Different priority levels (voice, video, data)
- Scheduling algorithms: WFQ (Weighted Fair Queuing), Priority Queuing

Packet Queue Types

- High priority: VoIP, video conferencing
- Medium priority: streaming video
- Best effort: web browsing, email

Queue Management

- Drop-tail: drop when full
- Random Early Detection (RED)
- Token bucket rate limiting

Complexity Analysis

Time and Space Complexity

Structure	Enqueue	Dequeue	Peek	Space
Queue (Array)	$O(1)$ amortized	$O(1)$	$O(1)$	$O(n)$
Queue (Linked List)	$O(1)$	$O(1)$	$O(1)$	$O(n)$ + pointers
Circular Queue	$O(1)$	$O(1)$	$O(1)$	$O(\text{capacity})$
Deque	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Priority Queue (Heap)	$O(\log n)$	$O(\log n)$	$O(1)$	$O(n)$

Special Cases

- For bounded integer priorities: Use bucket queues for $O(1)$ operations
- For monotonic priorities: Consider monotonic queue optimization
- For small fixed priorities: Array of queues (one per priority level)

Queue vs Stack

Queue vs Stack Comparison

Property	Queue	Stack
Principle	FIFO (First In, First Out)	LIFO (Last In, First Out)
Insert	Rear (enqueue)	Top (push)
Remove	Front (dequeue)	Top (pop)
Real-world	Waiting line	Plate stack
Use case	Scheduling	Recursion, undo

Queue Applications

- Breadth-First Search (BFS)
- Task scheduling
- Buffering
- Order processing

Stack Applications

- Depth-First Search (DFS)
- Expression evaluation
- Function calls
- Undo operations

Summary

Key Takeaways

Queue Fundamentals

- FIFO data structure: First In, First Out
- Essential operations: enqueue (rear), dequeue (front)
- All basic operations are $O(1)$ time complexity

Queue Variants

- **Circular Queue:** Efficient space usage with wrap-around
- **Deque:** Double-ended queue for flexible insertion/deletion
- **Priority Queue:** Element ordering based on priority (heap-based)

Important Applications

- Producer-consumer pattern and bounded buffers
- OS scheduling (CPU, I/O, process management)
- Network packet queuing and QoS

Thank You!

Questions?